Tabular Datasets for posttraining pipeline of Apertus

Semester Project Report — Spring 2026

Dijana Janjetovic Supervisor: Prof. Antoine Bosselut

June 2026

Abstract

Due to its ubiquitous nature, tabular data constitutes a new modality in the continued-pretraining mixture of Apertus, the open-weight Swiss large language model (LLM) [18]. Since a language model consumes flat token sequences, with tables inherent complex yet flexible infrastructure, they must first be serialized into text while preserving all its embedded information. Prior work on serialization addresses fine-tuning and in-context learning rather than pretraining at scale [3, 4, 5, 6]. We present a controlled ablation over five serialization formats—key:value, CSV, Markdown, JSON, and a novel SQL-schema-annotated format—in which data, architecture, random seed, and data order are held consistent. For each format, a 1.54B-parameter proxy model is trained on approximately 48B tokens of serialized tables from the T4 corpus [1] on CSCS Alps, and evaluated with a BLiMP-style minimal-pair protocol [8] comprising 1,475 statement pairs over five task types. CSV and JSON rank first and second under all conditions; on the subset unaffected by context-window truncation, CSV attains 58.4% against 55.2% for JSON, while Markdown falls below chance (47.5%). Together with CSV's approximately twofold token-efficiency advantage over record-style formats, the evidence supports CSV as the serialization format for Apertus tabular pretraining. Hence, the complete T4 corpus has been serialized and tokenized accordingly. We conclude with the further analyses required to consolidate these findings.

1 Introduction

Apertus is an open-weight LLM developed by the Swiss AI Initiative and trained on the Alps supercomputer at CSCS [18]. Its forthcoming iteration extends posttraining with additional modalities, including structured tabular data. Tables underlie a substantial fraction of real-world information—databases, spreadsheets, financial and scientific records—and reliable reasoning over them enables a correspondingly broad class of downstream tasks.

Incorporating tables into a text-based pretraining pipeline raises a deceptively simple question: in what textual form should a table be presented to the model? A transformer observes a one-dimensional token sequence rather than a two-dimensional grid, and the serialization format therefore determines which structural cues (column boundaries, header semantics, row delimitation) are available to be learned. The choice is consequential, as downstream performance on tabular benchmarks varies by several percentage points depending on the alignment between inference-time input formats and formats encountered during training [6].

Existing studies of table serialization concern few-shot classification, fine-tuning, or in-context learning [3, 4, 5, 7], and corpora derived from TabLib [2], such as T4 [1], serialize tables as classification-task wrappers unsuited to general next-token posttraining. To our knowledge, no controlled comparison of serialization formats for *pretraining* has been published. This project addresses that gap. Our contributions are: (i) a reproducible pipeline on CSCS Alps that converts raw T4 Parquet tables into Megatron-ready training data in five formats, including a segmentation scheme (*tabular chunking*) that retains every row of every table; (ii) a novel SQL-schema-annotated serialization for pretraining, derived at negligible cost from Parquet

metadata; (iii) a controlled five-way ablation with 1.54B-parameter proxy models trained under identical conditions; (iv) a minimal-pair evaluation across five tabular task types with an analysis of truncation artifacts; and (v) a format recommendation together with a fully serialized and tokenized T4 corpus ready for the Apertus data mixture.

2 Related Work

TabLib [2] comprises 627M tables extracted from GitHub and Common Crawl; T4 [1] filters it to approximately 3.1M high-quality tables (roughly 100B tokens), handling deduplication, PII removal, and language filtering, but leaving tables as raw two-dimensional cell values. Serialization templates for LLMs were introduced primarily for few-shot classification; TabLLM [3] established the key:value sentence template, and subsequent studies compare templates in fine-tuning and in-context settings [4, 5, 7]. `CREATE TABLE` schema prompts are standard in the text-to-SQL literature [14, 6], where the survey of Wu et al. [6] notes that the effectiveness of alternative table representations beyond retrieval-style tasks remains under-investigated; their use as a pretraining serialization is unexplored. Our evaluation adapts BLiMP [8], which scores minimal pairs by model likelihood without fine-tuning, and our proxy-model methodology follows the scaling-law practice of selecting design choices at small scale [9].

3 Method

3.1 Serialization formats

Five formats were selected to span the spectrum from compact to verbose and from implicit to explicit structure. Illustrated on a small example table:

A — key:value (baseline [3, 1]).

```
The city is Zurich. The population is 0.42. The area is 88.
The city is Geneva. The population is 0.20. The area is 16.
```

B — CSV.

```
city, population, area
Zurich, 0.42, 88
Geneva, 0.20, 16
```

C — Markdown. Pipe-delimited rows with a separator line, the form most frequently encountered during text training (GitHub, Wikipedia); alignment padding and the separator row incur token overhead.

```
| city   | population | area |
|--------|------------|------|
| Zurich | 0.42       | 88   |
| Geneva | 0.20       | 16   |
```

D — JSON.

```
 [{"city": "Zurich", "population": 0.42, "area": 88},
 {"city": "Geneva", "population": 0.20, "area": 16}]
```

F — SQL schema (novel format).

```
CREATE TABLE data (
    city VARCHAR,
    population FLOAT,  -- range: 0.13 to 0.42
    area INT           -- range: 16 to 88
);
city, population, area
Zurich, 0.42, 88
Geneva, 0.20, 16
```

A `CREATE TABLE` header with column types and value ranges, inferred directly from Parquet column statistics, precedes compact CSV rows. The format exploits the model's prior exposure to SQL [14] and is hypothesized to be most beneficial for tables whose column names carry little semantic content (ie. `col_1`, `value`), which survive T4's filtering in non-trivial numbers.

3.2 Serialization design

In contrast to T4's one-row-per-sample classification setup [1], each training sample packs as many rows of a single table as fit a 3,800-token budget (the remainder of the 4,096-token context being reserved for special tokens), enabling the model to learn cross-row regularities. Tables exceeding the budget are split into multiple segments by greedy row accumulation with per-row token pre-computation (*tabular chunking*); every segment repeats the header or schema, and no row is discarded. Rows are shuffled deterministically with a filename-derived seed so that all five format variants observe identical row orderings, removing data order as a confound. Cleaning removes artifact columns (`Unnamed:*`, `index`), all-null columns, and columns containing cells longer than 500 characters.

3.3 Pipeline and training setup

The pipeline executes on CSCS Alps (Clariden) in four phases: (1) serialization of T4 chunks 0–23 via SLURM array jobs, yielding approximately 48B tokens per format; (2) tokenization with the Apertus BPE tokenizer (128k vocabulary) into Megatron binary format using datatrove's `MegatronDocumentTokenizer`; (3) training of five proxy models with Megatron-LM [10]; and (4) evaluation on held-out chunks 24–25. Two operational findings merit documentation: Clariden compute nodes lack internet access, so the tokenizer must be pre-cached on the login node, and Megatron's `preprocess_data.py` silently produces empty output under this restriction, whereas datatrove with a cached tokenizer path does not.

Since training Apertus itself five times would be computationally prohibitive; following standard scaling-law methodology [9], the ablation study employs proxy models of the same architectural family, under the assumption that the relative ranking of formats transfers across scale. Table 1 summarizes the configuration; all five runs share seed, schedule, and data order, leaving serialization format as the sole independent variable.

Architecture		**Training**	
Layers	32	Tokens per run	∼48B (28,610 steps)
Hidden / FFN size	2,048 / 6,144	Global batch	256 × 4,096 tokens
Heads	16 (GQA [12], 4 KV)	Optimizer	Adam (β_1=0.9, β_2=0.95)
Parameters	1.54B + 524M emb.	Learning rate	3e−4 cosine → 3e−5
Positional encoding	RoPE [11]	Precision	bfloat16
Normalization	RMSNorm	Hardware	8 nodes, 32× GH200, ∼12 h

Table 1: Proxy-model architecture and training configuration.

3.4 Evaluation protocol

From held-out chunks, 295 tables were sampled (each with at least two rows and a header fitting the token budget in all five formats). For each table, a context prompt is constructed within the 3,800-token budget, and LLaMA-3.1-8B-Instruct [13] was prompted to generate one factually correct statement and one plausible distractor for each of five task types—lookup, aggregation, comparison, inference, and multihop—yielding $295 \times 5 = 1{,}475$ minimal pairs per format. Identical statement pairs are used across formats, so generation noise cannot bias the ranking. Each proxy model scores both statements by mean negative log-likelihood (NLL) conditioned on the table context; a pair is correct when the true statement receives the higher likelihood. The protocol is a binary forced choice (chance: 50%) requiring neither labels nor fine-tuning, in the manner of BLiMP [8].

4 Results

Token efficiency. Measured over 200 sampled tables (Table 2), CSV and SQL schema accommodate approximately twice as many rows per sample as key:value and JSON; the schema header costs only ∼3% of rows relative to plain CSV. For a fixed pretraining token budget, higher row density translates directly into more unique tabular content observed by the model.

Format	**Avg. rows/sample**	**Relative to key:value**
key:value (baseline)	30.7	1.00×
JSON	29.9	0.97×
Markdown	50.9	1.66×
SQL schema	58.2	1.90×
CSV	59.9	1.95×

Table 2: Token efficiency over 200 tables (context 4,096; budget 3,800).

Training dynamics. All runs converge stably over ∼29k steps (Figure 1). The verbose formats (JSON, key:value, Markdown) attain *lower* final training loss than CSV and SQL schema. This reflects the greater predictability of repetitive token streams rather than superior table understanding; training loss is therefore not a valid selection criterion, motivating the downstream evaluation. To clarify, the CSV spike near step 4,000 is a checkpoint-reload artifact that recovers within 200 steps.

Minimal-pair accuracy. Table 3 reports accuracy on the full evaluation set. CSV and JSON rank first and second overall, separated by less than one percentage point, which is indistinguishable with a single run per format, while key:value and Markdown fall below chance.

Format	Lookup	Aggreg.	Compar.	Infer.	Multihop	**Overall**
JSON	44.9	46.5	59.8	**54.2**	**58.8**	**52.8**
CSV	45.6	46.5	**62.8**	49.3	57.4	52.3
SQL schema	**47.8**	**50.4**	57.8	49.0	55.3	52.0
key:value	40.8	47.9	58.1	46.9	53.2	49.3
Markdown	44.0	45.1	56.5	48.6	51.1	49.0

Table 3: Minimal-pair accuracy (%) on the full evaluation set (1,475 pairs per format; chance = 50%).

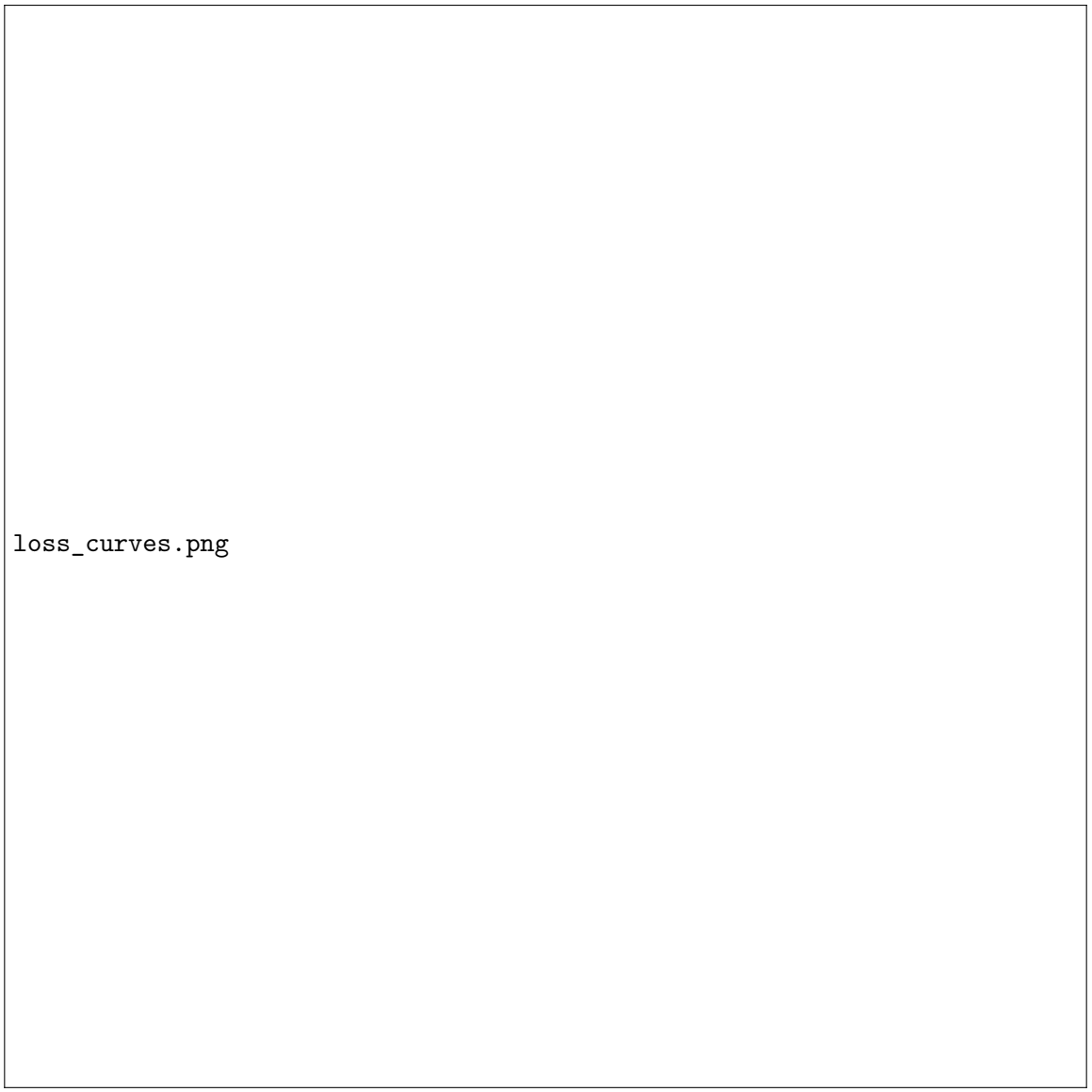

Figure 1: Training loss by serialization format. Verbose formats yield lower loss because their token streams are more predictable, not because the resulting models are better.

Truncation artifact. Tables exceeding the 4,096 context window are truncated, so the cell required to verify a statement may be absent from the prompt. This depresses lookup and aggregation below chance *for all formats* and is a property of the evaluation rather than of any format. On the clean subset of ∼147 pairs whose tables fit entirely in context (the more reliable condition) CSV attains 58.4%, JSON 55.2%, and Markdown 47.5%, ie. below chance.

Task-type effects. Comparison is the strongest category for every format (56–63%). SQL schema is the only format above chance on aggregation (50.4%), consistent with its explicit type declarations. JSON leads on inference and multihop, where per-row repetition of column names plausibly supports multi-step reasoning.

5 Discussion

Recommendation. Three converging observations support serializing Apertus tabular data as CSV. First, CSV performs best under the truncation-free condition (+3.2 points over JSON).

Second, its ranking is stable across all conditions, whereas JSON's full-set advantage does not persist on the clean subset. Third, CSV is approximately twice as token-efficient as JSON and key:value (Table 2), which at a posttraining budget of hundreds of billions of tokens roughly doubles the unique tabular content seen during training. Overall, the result is counterintuitive, with the least structurally explicit format performing the best, and the inverse relationship between training loss and downstream accuracy cautions generally against format selection by loss.

Limitations. These results constitute current findings rather than final conclusions. With one training run per format, no variance estimates exist for margins below approximately three points (CSV vs. JSON in particular). The truncation artifact, though format-neutral, suppresses the signal in two of five task categories. Pairs are aggregated per table with equal weight and scored by mean per-token NLL; both choices interact with statement length and table size. Distractors are LLM-generated and shared across formats, leaving the ranking unbiased but tying absolute accuracies to distractor difficulty. Finally, the transfer of format rankings from 1.54B parameters to Apertus scale, while standard practice [9], is untested here, and known serialization edge cases remain (redundant geometry columns passing the cell filter; semantically opaque range annotations for epoch-style timestamps).

Future work. Feedback from the project presentation yields four immediate refinements to the evaluation: (i) removing from each prompt the parts of the table irrelevant to the statement, so that the context budget is spent where it matters; (ii) quantifying how often the ground-truth answer is actually present in the (possibly truncated) context, and excluding or separately analyzing pairs where it is not; (iii) truncating tables *before* statement generation, so that every statement is answerable from the visible context by construction; and (iv) complementing per-table scoring with length-controlled, per-token analyses. Subsequent steps include rerunning the ablation at an 8,192-token context to completely eliminate truncation at the source, training multiple seeds per format to attach confidence intervals to the CSV–JSON margin, closing the gap between T4's ∼100B tokens and the continued-posttraining target of 300B tokens with additional sources such as GitTables [15], and ultimately evaluating Apertus after continued pretraining on WikiTableQuestions [16] and TabFact [17], with contamination checks against the evaluation benchmarks.

6 Conclusion

This project provides, to our knowledge, the first controlled comparison of table serialization formats for language-model pretraining: five formats, five identical 1.54B-parameter training runs of ∼30B tokens each, and a 1,475-pair minimal-pair evaluation. CSV emerges as the recommended format, leading the truncation-free evaluation (58.4%), ranking consistently across conditions, and offering the highest token efficiency, while formats that minimize training loss generally transfer worst. The full T4 corpus is serialized and tokenized in CSV format and ready for integration into the Apertus continued-pretraining mixture; the refinements outlined above will determine the robustness of the CSV–JSON margin before the full-scale run.

Acknowledgments

I thank my supervisor and the NLP team involved in Apertus for their guidance throughout the semester, in particular Ayush and Aditi for the discussions that shaped the constraints of this work, and Clara for her feedback on the evaluation design.

References

[1] J. Gardner, J. C. Perdomo, L. Schmidt. Large scale transfer learning for tabular data via language modeling. *NeurIPS*, 2024. arXiv:2406.12031.

[2] G. Eggert, K. Huo, M. Biderman, J. Waugh. TabLib: A dataset of 627M tables with context. arXiv:2310.07875, 2023.

[3] S. Hegselmann, A. Buendia, H. Lang, M. Agrawal, X. Jiang, D. Sontag. TabLLM: Few-shot classification of tabular data with large language models. *AISTATS*, 2023. arXiv:2210.10723.

[4] S. Jaitly, T. Shah, A. Shugani, R. S. Grewal. Towards better serialization of tabular data for few-shot classification with large language models. arXiv:2312.12464, 2023.

[5] X. Wang et al. Table meets LLM: Can large language models understand structured table data? A benchmark and empirical study. *WSDM*, 2024. arXiv:2305.13062.

[6] Wu et al. Tabular data understanding with LLMs: A survey of recent advances and challenges. arXiv:2508.00217, 2025.

[7] K. Ono, A. Lee. Text serialization and their relationship with the conventional paradigms of tabular machine learning. arXiv:2406.13846, 2024.

[8] A. Warstadt, A. Parrish, H. Liu, A. Mohananey, W. Peng, S.-F. Wang, S. R. Bowman. BLiMP: The benchmark of linguistic minimal pairs for English. *TACL*, 8:377–392, 2020.

[9] J. Hoffmann et al. Training compute-optimal large language models. *NeurIPS*, 2022. arXiv:2203.15556.

[10] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, B. Catanzaro. Megatron-LM: Training multi-billion parameter language models using model parallelism. arXiv:1909.08053, 2019.

[11] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, Y. Liu. RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 2024. arXiv:2104.09864.

[12] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, S. Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. *EMNLP*, 2023. arXiv:2305.13245.

[13] A. Grattafiori et al. The Llama 3 herd of models. arXiv:2407.21783, 2024.

[14] T. Yu et al. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. *EMNLP*, 2018. arXiv:1809.08887.

[15] M. Hulsebos, Ç. Demiralp, P. Groth. GitTables: A large-scale corpus of relational tables. *SIGMOD*, 2023. arXiv:2106.07258.

[16] P. Pasupat, P. Liang. Compositional semantic parsing on semi-structured tables. *ACL*, 2015. arXiv:1508.00305.

[17] W. Chen et al. TabFact: A large-scale dataset for table-based fact verification. *ICLR*, 2020. arXiv:1909.02164.

[18] Swiss AI Initiative. Apertus. `https://huggingface.co/swiss-ai/Apertus-70B-2509`, 2025.